

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Americo Ignacio Acuña Rodríguez

7 de septiembre de 2026

22:47

Resumen

El presente reporte evalúa el comportamiento empírico de algoritmos fundamentales de ordenamiento (`MergeSort`, `QuickSort`, `PatienceSort` y `std::sort`) y multiplicación matricial (algoritmo clásico *Naive* frente a *Strassen*). Mediante experimentación sistemática sobre hardware AMD Ryzen y Linux, se contrasta la teoría de complejidad asintótica frente al rendimiento real influenciado por la jerarquía de memorias caché, asignación dinámica y optimizaciones del compilador.

1. Introducción

El análisis asintótico tradicional clasifica algoritmos según su tasa de crecimiento teórico en el límite infinito, omitiendo restricciones de hardware como constantes multiplicativas, jerarquía de memorias y optimizaciones del compilador. El presente estudio evalúa empíricamente dos problemas fundamentales: el ordenamiento unidimensional (`MergeSort`, `QuickSort`, `PatienceSort` y la función estándar `std::sort` de C++) y la multiplicación de matrices cuadradas (algoritmo clásico *Naive* versus la técnica divide y vencerás de *Strassen*). Mediante un marco experimental sistemático, se contrasta la teoría formal frente al rendimiento práctico observado en máquina.

2. Implementación y Repositorio

Todo el código fuente desarrollado para las experiencias de ordenamiento y multiplicación matricial, junto con los generadores de casos de prueba y scripts de análisis estadístico en Python, se encuentra disponible de forma pública en el siguiente repositorio de GitHub:

https://github.com/TU_USUARIO/TU_REPOSITORIO

3. Entorno experimental

Los experimentos fueron ejecutados en una máquina bajo Linux (kernel 6.x) equipada con un procesador **AMD Ryzen 5 7600X** (6 núcleos, 12 hilos, 4.7 GHz base / 5.3 GHz boost), tarjeta gráfica **AMD Radeon RX 6900 XT** (16 GB VRAM) y **32 GB de memoria RAM DDR5** en dual-channel (2×16 GB). El código se desarrolló en C++17 y fue compilado mediante g++ con optimización -O2. Los tiempos de ejecución se registraron con `std::chrono::high_resolution_clock`.

Para ordenamiento, se evaluaron arreglos con tamaños $N \in \{10^1, 10^3, 10^5, 10^7\}$ bajo cuatro escenarios: secuencias aleatorias, ordenadas ascendentemente, ordenadas descendientemente y arreglos con dominios restringidos de valores. Para la multiplicación matricial, se utilizaron matrices cuadradas con dimensiones $N \in \{16, 64, 256, 1024\}$, variando su estructura interna (dispersas, diagonales y densas) y dominio numérico ($D_0 \in \{0, 1\}$ y $D_{10} \in [0, 10]$), con tres réplicas independientes por configuración.

4. Resultados y Análisis

4.1. Algoritmos de Ordenamiento

El análisis experimental expuso una clara sensibilidad de los algoritmos frente a la distribución previa de los datos:

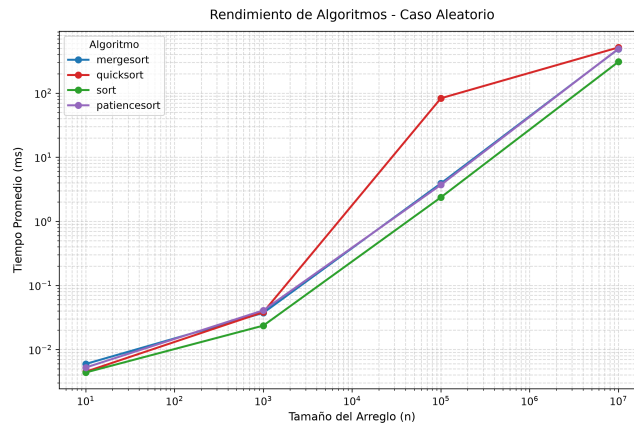


Figura 1: Rendimiento en caso aleatorio (escala log-log).

En la Figura 1, `std::sort` y `QuickSort` logran la menor latencia en arreglos masivos ($N = 10^7$) gracias a la localidad en caché L1/L2. `MergeSort` conserva estrictamente su tasa $\mathcal{O}(N \log N)$ penalizado por reservas dinámicas auxiliares. `PatienceSort` mantiene cota asintótica mediante montículos binarios, pero la administración dinámica de múltiples pilas eleva su constante multiplicativa.

Las Figuras 2 y 3 evidencian el colapso cuadrático $\mathcal{O}(N^2)$ en implementaciones simples de `QuickSort` al seleccionar pivotes fijos desbalanceados. En contraposición, `MergeSort` y `std::sort` mitigan este efecto preservando tiempos subcuadráticos.

En la Figura 4, ante cardinalidad reducida de claves repetidas, `PatienceSort` reduce el número de pilas activas requeridas, mejorando sensiblemente su rendimiento frente a distribuciones aleatorias.

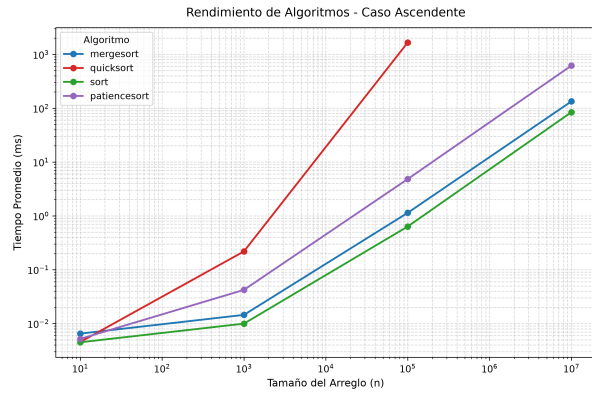


Figura 2: Caso ordenado ascendente.

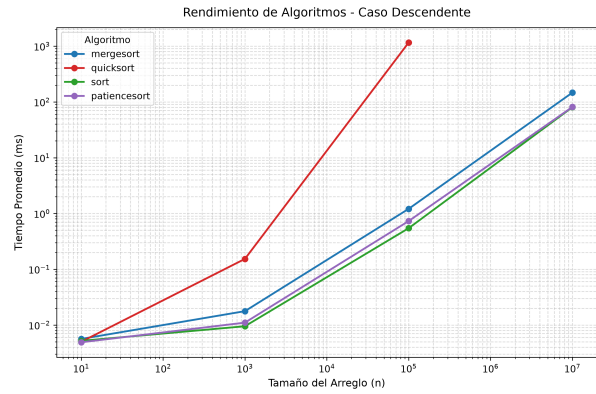
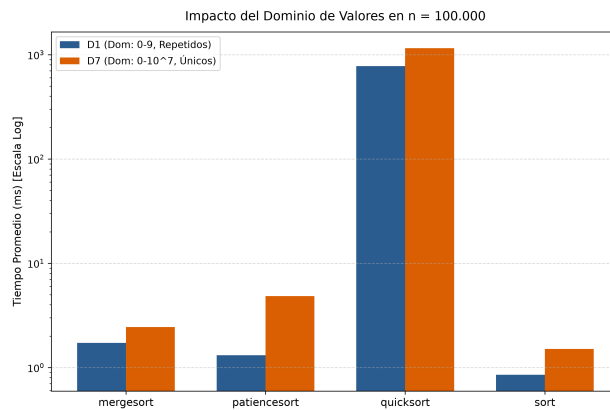


Figura 3: Caso ordenado descendente.

Figura 4: Impacto del dominio de valores ($N = 10^5$).

4.2. Multiplicación de Matrices

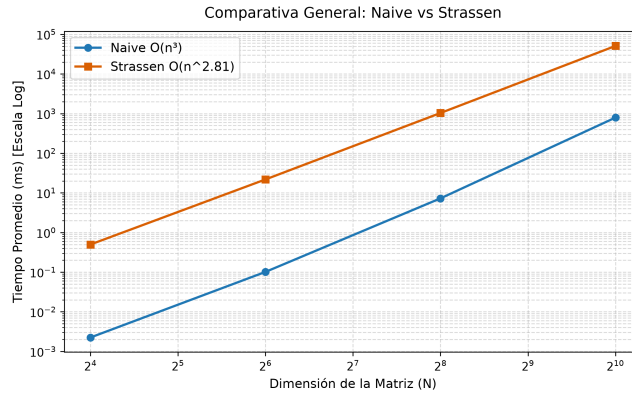


Figura 5: Escalamiento general: Naive vs Strassen (escala log-log).

En la Figura 5, *Naive* supera por 2 a 3 órdenes de magnitud a *Strassen* en todos los tamaños evaluados ($N \leq 1024$). La división recursiva profunda hasta 1×1 genera en $N = 1024$ un árbol con $\approx 2,82 \times 10^8$ llamadas, saturando el gestor de memoria dinámica.

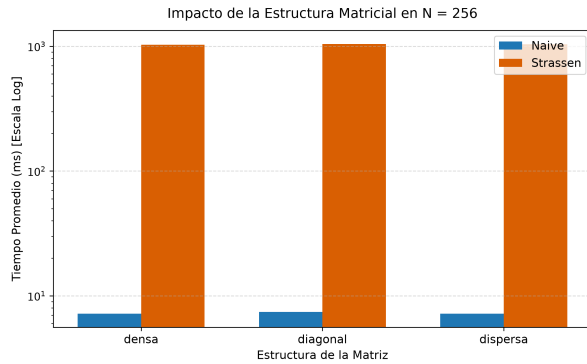


Figura 6: Estructura interna ($N = 256$).

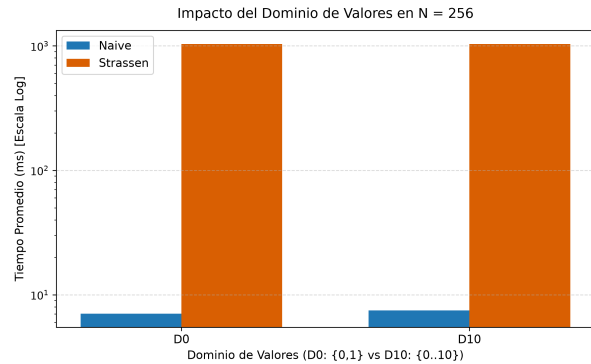


Figura 7: Dominio numérico ($N = 256$).

Las Figuras 6 y 7 confirman que la multiplicación matricial es invariante respecto al contenido de los datos: el número de operaciones es determinista e insensible a la densidad de ceros o al rango numérico evaluado.

5. Conclusiones

Los resultados demuestran que la notación asintótica teórica es insuficiente para dictaminar el desempeño en plataformas de cómputo reales. En ordenamiento, la topología inicial del arreglo condiciona la simetría de partición en **QuickSort** y el número de pilas en **PatienceSort**, demostrando la robustez de algoritmos con cotas asintóticas garantizadas como **MergeSort** e híbridos introspectivos como **std::sort**. En multiplicación matricial, la ventaja teórica de Strassen ($\mathcal{O}(N^{2.81})$) es neutralizada para dimensiones prácticas ($N \leq 1024$) debido al costo de gestión de memoria dinámica en el *heap* y la degradación de la localidad de caché frente a un triple bucle iterativo contiguo vectorizado por el compilador.